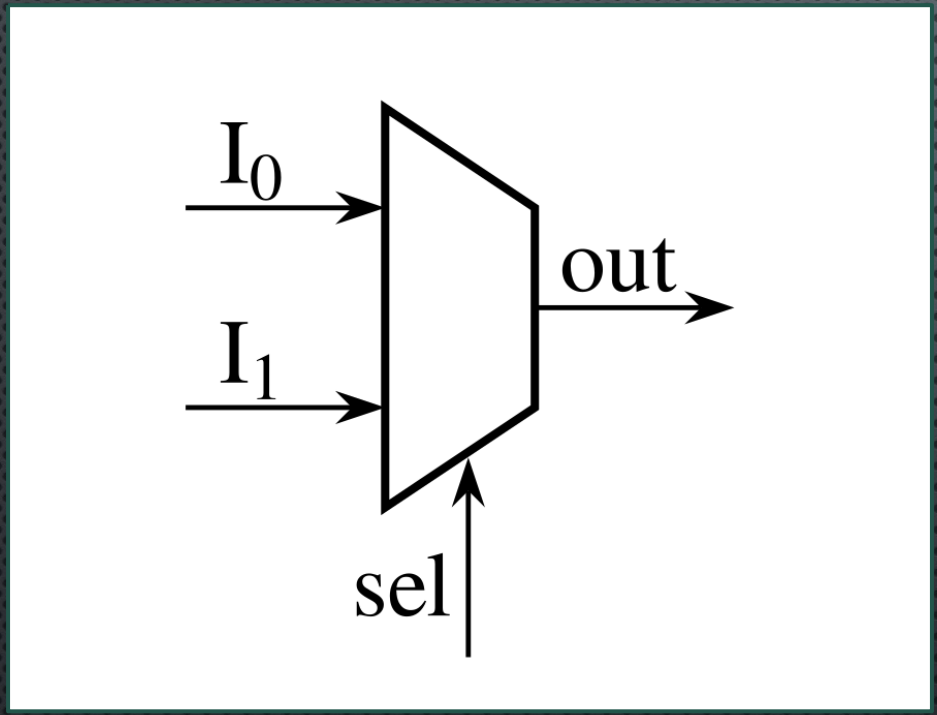
DIGITAL SYSTEMS

DESIGN AND VERIFICATION

MULTIPLEXERS



A	B	S	Out
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

```
module mux2_dataflow
( input a, b, sel,
   output y );

 assign y = (sel) ? b : a;

 endmodule
```

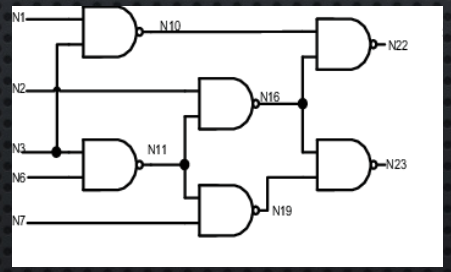
Conditional Operator

condition ? value_if_true : value_if_false

MUX - 2 TO 1 (DIFFERENT DESIGN STYLES)

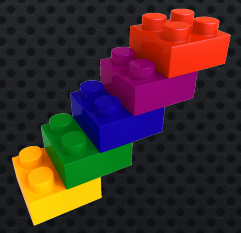
Simple Combinational Logic: **assign** keyword and **Boolean operators** (&, |, ^, ~)

```
module mux2_dataflow
( input a, b, sel,
   output y );

 assign y = (sel) ? b : a;

 endmodule
```



Manually instantiate modules and connect them using wire

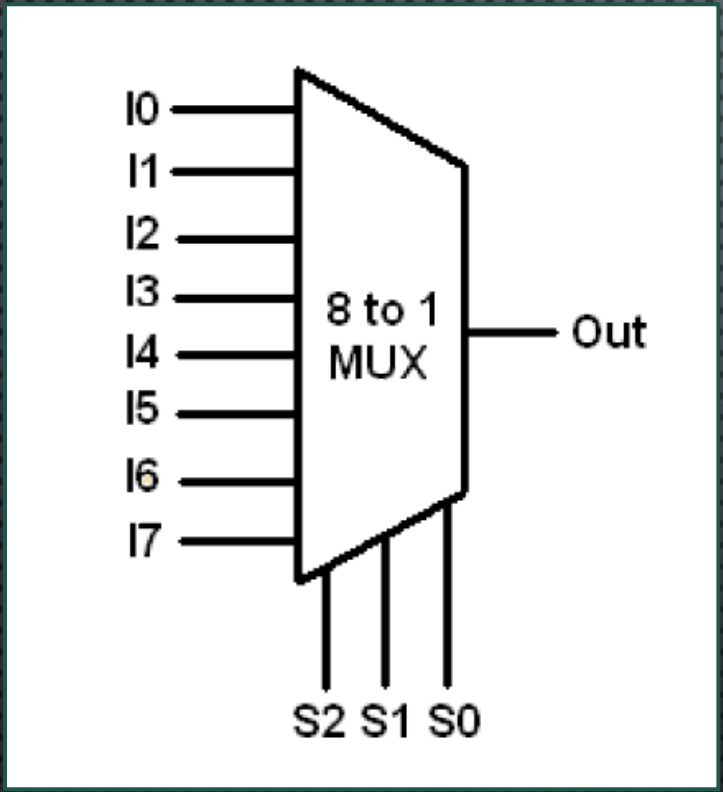
```
module mux2_structural (
    input a, b, sel,
    output y
);
    wire not_sel, w1, w2;
    not (not_sel, sel);
    and (w1, a, not_sel);
    and (w2, b, sel);
    or  (y, w1, w2);

endmodule
```



Always blocks and **if/else** or **case** statements. (Complex Logic)

```
module mux2_behavioral (
    input a, b, sel,
    output reg y
);
    always @(*) begin
        if (sel)
            y = b;
        else
            y = a;
    end
endmodule
```

MULTIPLEXERS
8 TO 1

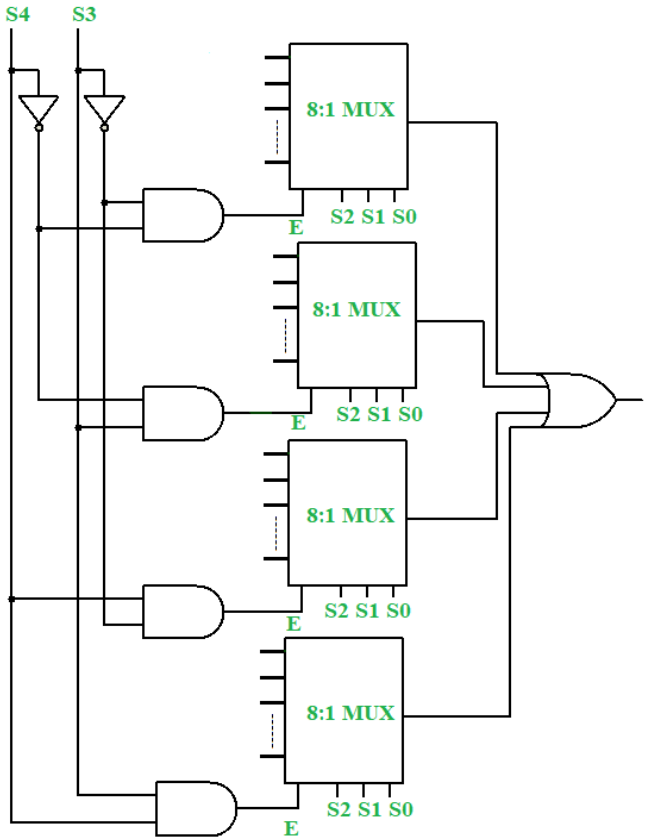



```
module mux8 (
    input [7:0] data_in,
    input [2:0] sel,
    output reg y
);
    always @(*) begin
        case (sel)
            3'b000: y = data_in[0];
            3'b001: y = data_in[1];
            3'b010: y = data_in[2];
            3'b011: y = data_in[3];
            3'b100: y = data_in[4];
            3'b101: y = data_in[5];
            3'b110: y = data_in[6];
            3'b111: y = data_in[7];
            default: y = 1'b0; // IMPORTANT
        case
    end
endmodule
```

PROBLEM !!!

In an audio dedicated FPGA, you have 32 entry channels and just one output to the. ¿How do you select which one to play without duplicating the logic?

Select inputs					Output=Y
S_4	S_3	S_2	S_1	S_0	
0	0	0	0	0	D_0
⋮		⋮	⋮	⋮	⋮
0	0	1	1	1	D_7
0	1	0	0	0	D_8
⋮		⋮	⋮	⋮	⋮
0	1	1	1	1	D_{15}
1	0	0	0	0	D_{16}
⋮		⋮	⋮	⋮	⋮
1	0	1	1	1	D_{23}
1	1	0	0	0	D_{24}
⋮		⋮	⋮	⋮	⋮
1	1	1	1	1	D_{31}



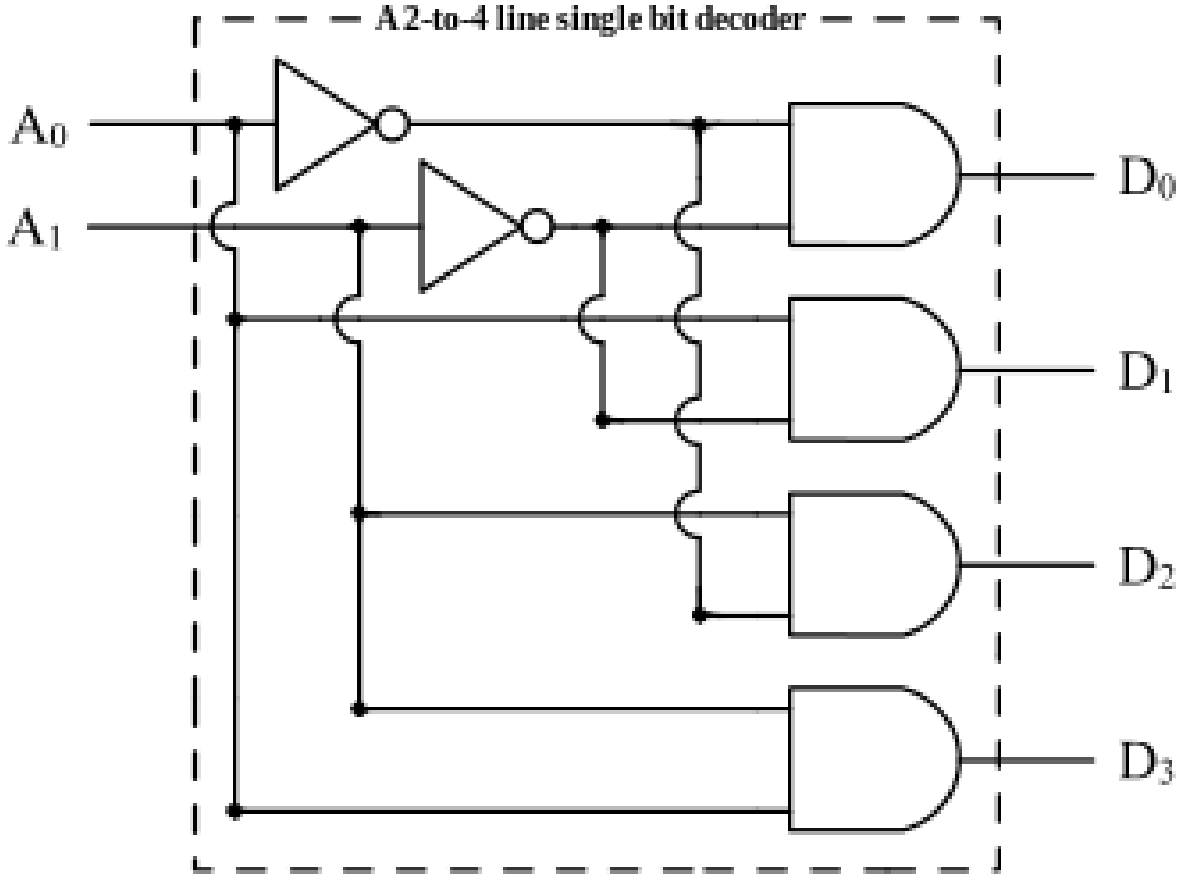
PROBLEM !!!

In an audio dedicated FPGA, you have 32 entry channels and just one output to the. ¿How do you select which one to play without duplicating the logic?

```
module mux32 (
    input  wire [31:0] D,
    input  wire S4, S3, S2, S1, S0,
    output wire Y
);

    // Wires for Enables
    wire E0, E1, E2, E3;

    // Wires for outputs of MUXes
    wire Y0, Y1, Y2, Y3;

    //
    assign E0 = (~S4 & ~S3);  // 00: MUX 0
    assign E1 = (~S4 &  S3);  // 01: MUX 1
    assign E2 = ( S4 & ~S3);  // 10: MUX 2
    assign E3 = ( S4 &  S3);  // 11: MUX 3
```

```
    mux8_en m0 (
        .D(D[7:0]),
        .S2(S2), .S1(S1), .S0(S0),
        .E(E0),
        .Y(Y0)
    );

    mux8_en m1 (
        .D(D[15:8]),
        .S2(S2), .S1(S1), .S0(S0),
        .E(E1),
        .Y(Y1)
    );

    (...)

    // OR Output
    assign Y = Y0 | Y1 | Y2 | Y3;

endmodule
```

DECODERS
2 TO 4



Truth Table

A_1	A_0	D_3	D_2	D_1	D_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Minterm Equations

$$D_0 = \overline{A_1} \cdot \overline{A_0}$$

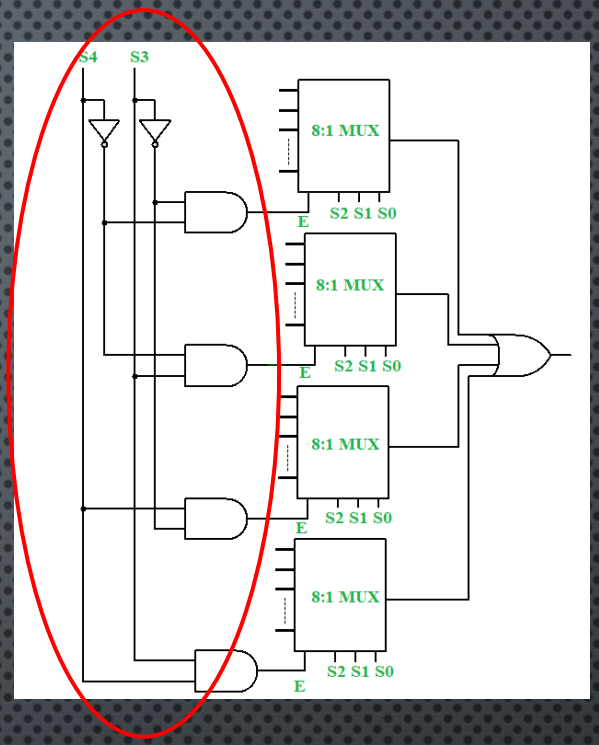
$$D_1 = \overline{A_1} \cdot A_0$$

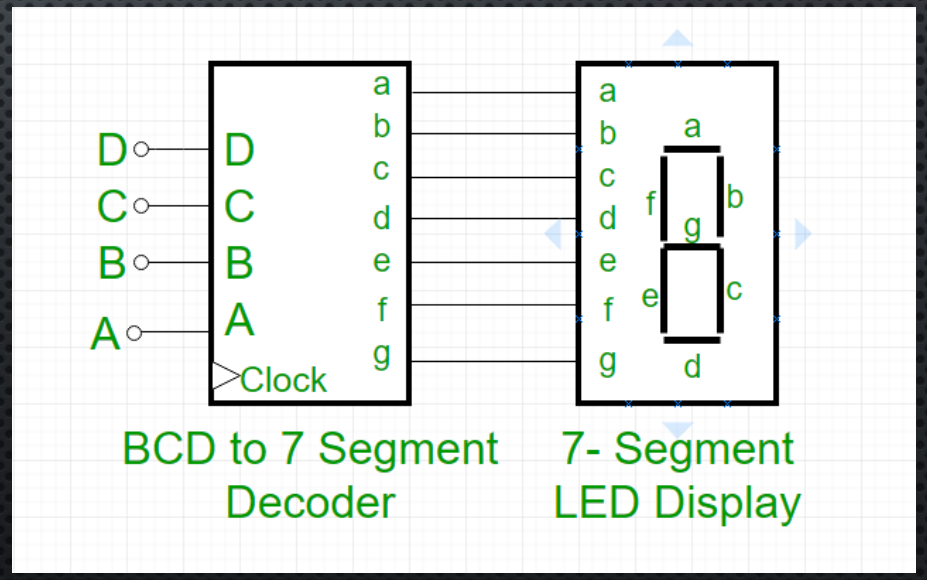
$$D_2 = A_1 \cdot \overline{A_0}$$

$$D_3 = A_1 \cdot A_0$$

DECODERS – PURPOSE ?

- Selección de chip en buses
- Control de Display de 7 Segmentos
- Demultiplexado de Señales



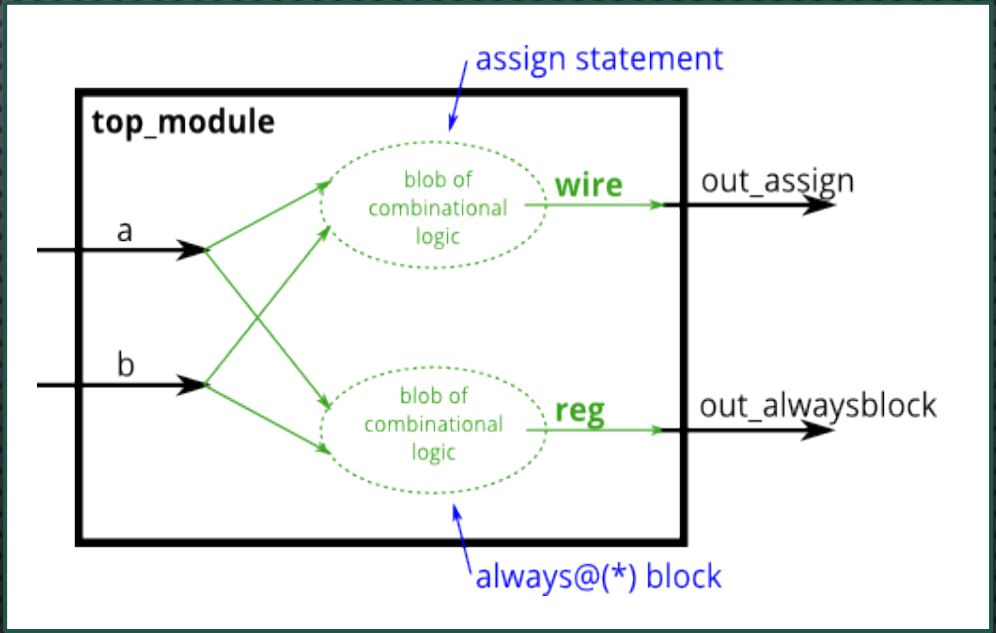


DECODERS (ALWAYS, ASSIGN)

Assign

- Continous logic assignments
- Wire type signals
- Immediately updated
- Direct Connections

```
assign out1 = a & b | c ^ d;
always @(*) out2 = a & b | c ^ d;
```



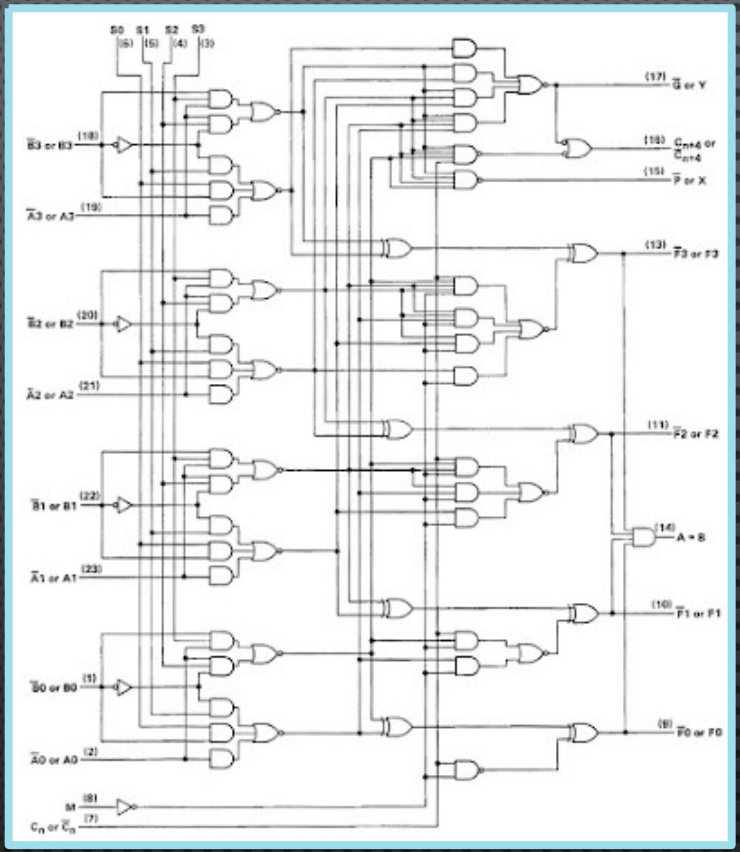
Always

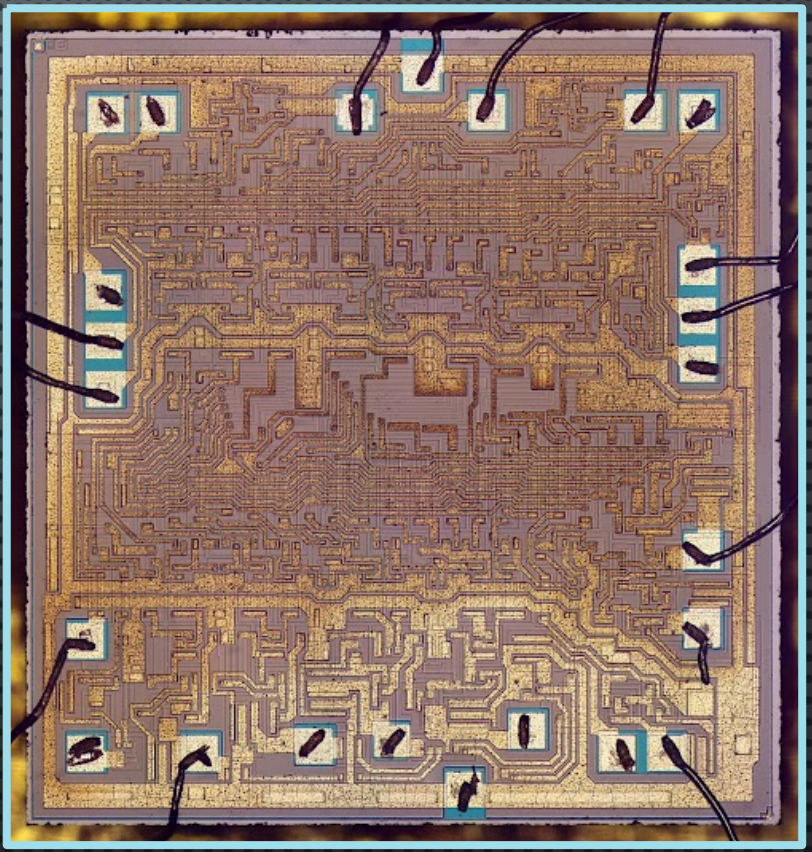
- Procedimental Block -> **Sequential Logic**
- Reg type signals
- Always @(*) // Always @(posedge clk)

ALU'S – LIL BIT OF HISTORY

Arithmetic/Logic Unit

- It makes all the math operations with binary information
- Every GPU/CPU has one
- Needed for processing





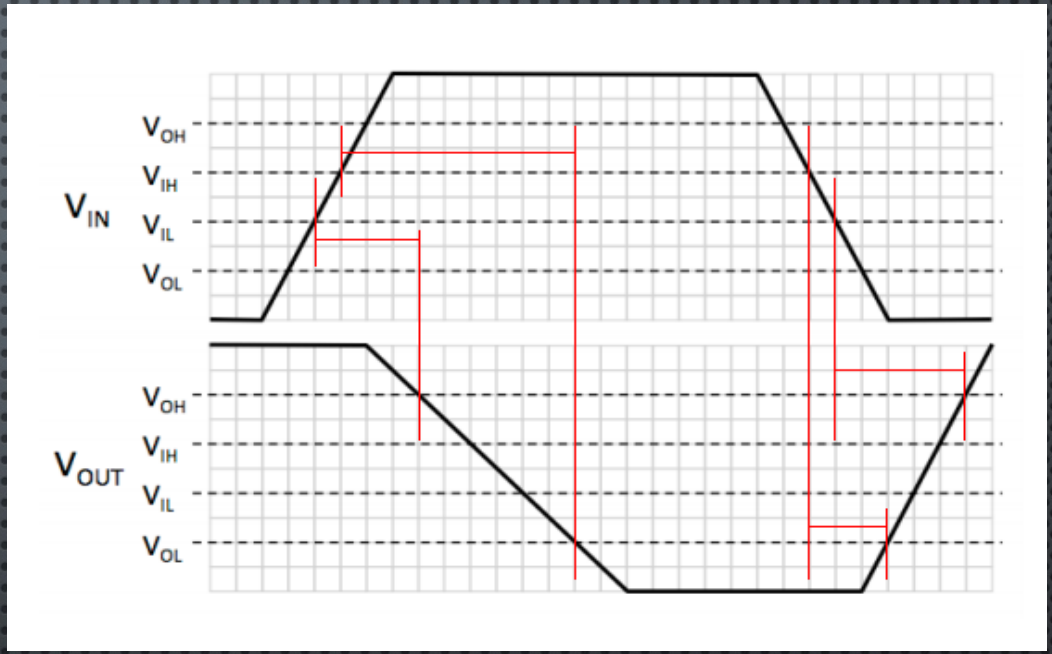
Famous: 74181
First ALU within a single chip

ALU – 4 BITS

```
module alu4 (
    input       [3:0] A,
    input       [3:0] B,
    input       [1:0] opcode,
    output reg [3:0] result
);

    // Procedural block
    always @(*) begin
        case (opcode)
            2'b00: result = A + B; // Full Adders **
            2'b01: result = A - B; // Resta
            2'b10: result = A & B; // AND
            2'b11: result = A | B;//  OR
            default: result = 4'b0000;
        endcase
    end

endmodule
```

PHYSICS REALITY -> PROPAGATION DELAY

- Sequential Logic -> Logic that is leaded by a clock **clock**
- We start dealing with -> Timing Errors



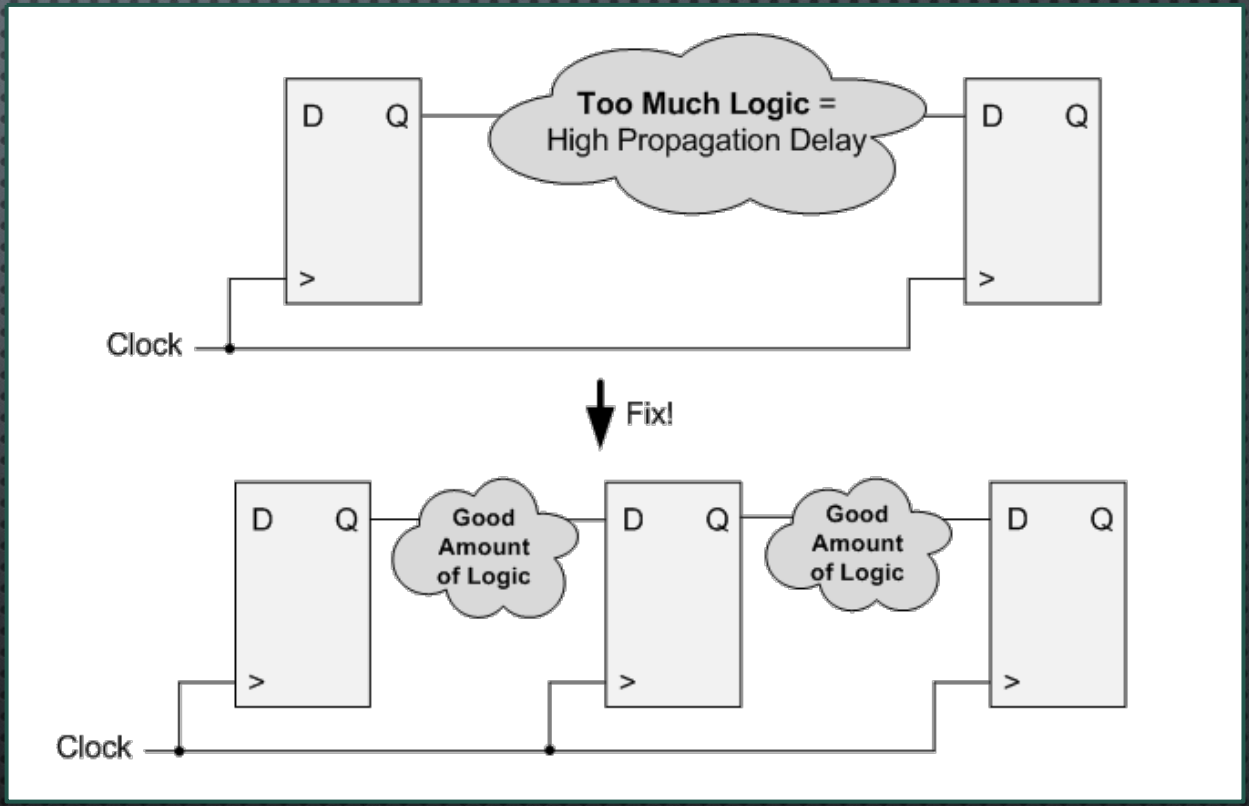
- If you have 2 Flip-Flops with 10 ns of propagation delay between them. If we use a 50 MHz clock, there should be no problem, however with 200 Mhz the circuit will start taking random behaviors.

50 MHz -> 20ns
- 20 ns – 10 ns = 10 ns

200 MHz -> 5ns
- 5 ns – 10 ns = **-5ns**

PHYSICS REALITY -> PROPAGATION DELAY

- Slow Down The Clock
- Pipelines !!!

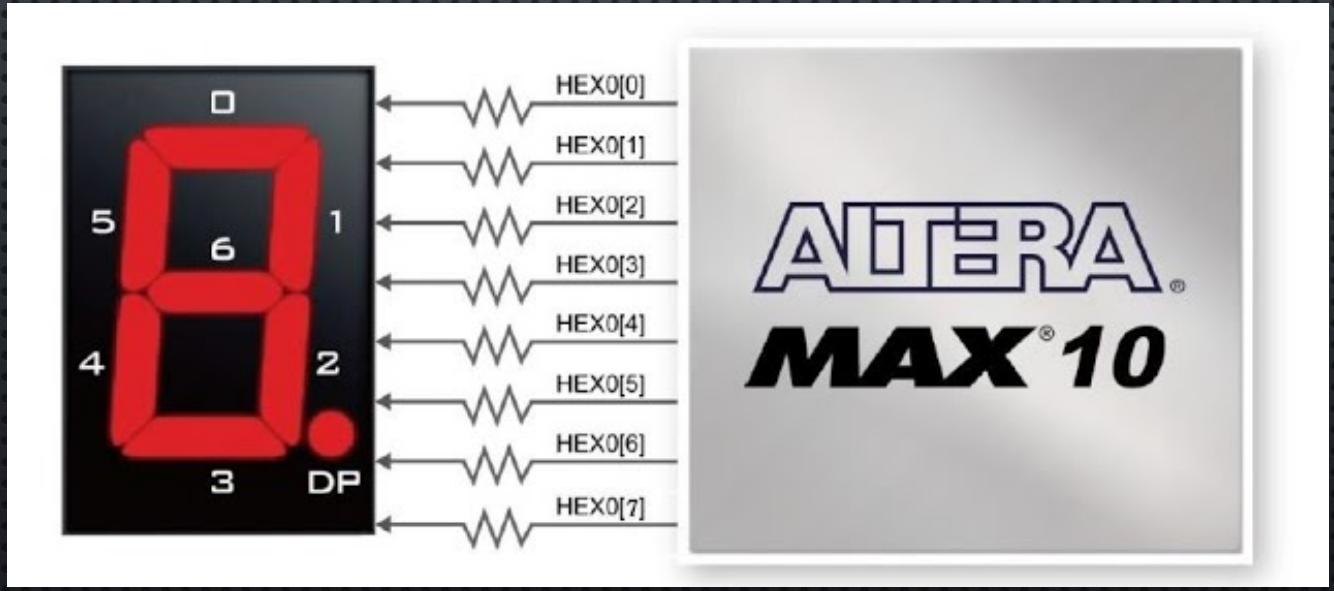


Diagramas de NandLand

7 SEGMENT DISPLAY

- In the DE10-Lite, the displays have and Common Anode config
- To turn them on you have to send a logical 0

0 = ON , 1 = OFF

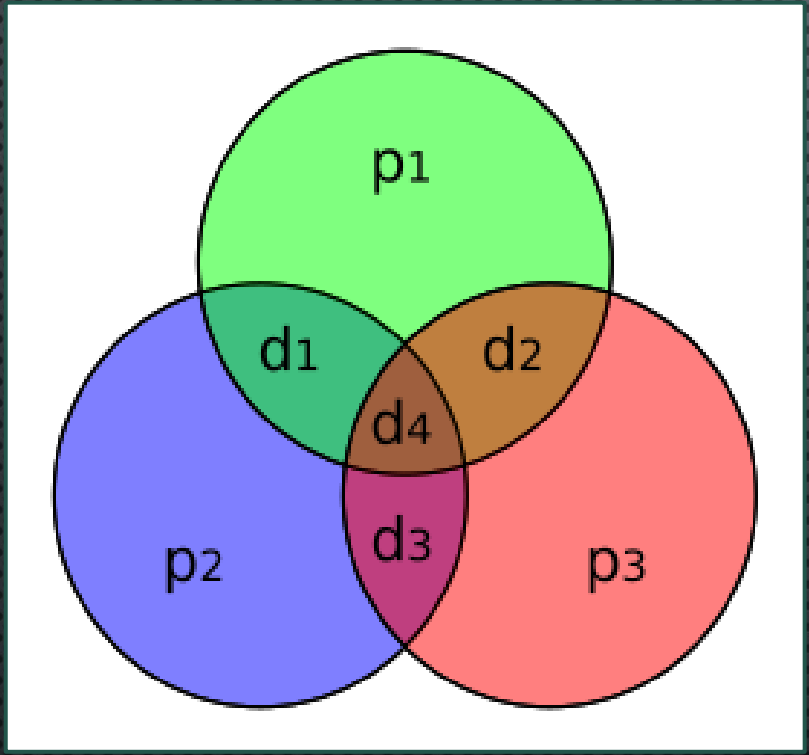


HEX TO 7 SEGMENT DECODER

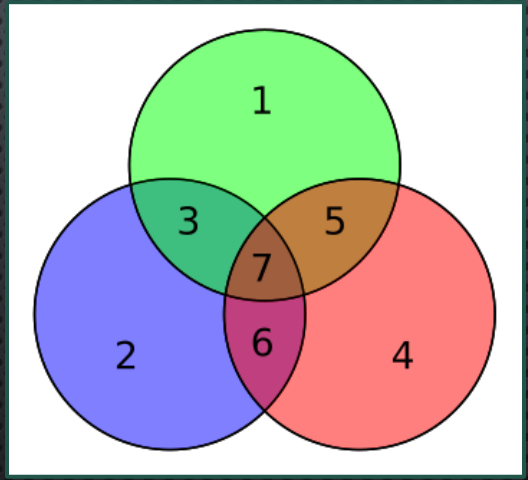
```
module hex_decoder (
    input       [3:0] hex_digit,
    output reg [6:0] segments

    always @(*) begin
        case (hex_digit)
            // Remember : 0 ON, 1 OFF. Order: g-f-e-d-c-b-a
            4'h0: segments = 7'b1000000;



            4'hF: segments = 7'b0001110; // F
            default: segments = 7'b1111111;
        endcase
    end

endmodule
```

```
module top_module (
    input  wire [9:0] SW,   // 10 switches
    output wire [7:0] HEX0 // 8 segs of the 7-seg dsply
);

    // Instantiate decoder
    hex_decoder mi_display (
        .hex_digit (SW[3:0]),
        .segments  (HEX0[6:0])
    );

    // TURN OFF the decimal point
    assign HEX0[7] = 1'b1;

endmodule
```

BONUS !!!!!!
HAMMING CODE



$$p_1 = d_1 \oplus d_2 \oplus d_4$$
$$p_2 = d_1 \oplus d_3 \oplus d_4$$
$$p_3 = d_2 \oplus d_3 \oplus d_4$$



position #:	1	2	3	4	5	6	7
	p_1	p_2	x_3	p_4	x_2	x_1	x_0

1 = (001)
2 = (010)
3 = (011)
4 = (100)
5 = (101)
6 = (110)
7 = (111)

BONUS !!!!!!
HAMMING CODE

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

position #:	1	2	3	4	5	6	7
	p_1	p_2	x_3	p_4	x_2	x_1	x_0
	1	1	0	0	1	1	0
	1	0	0	0	1	1	0

$c_1 = p_1 \text{ XOR } x_3 \text{ XOR } x_2 \text{ XOR } x_0$

$c_2 = p_2 \text{ XOR } x_3 \text{ XOR } x_1 \text{ XOR } x_0$

$c_4 = p_4 \text{ XOR } x_2 \text{ XOR } x_1 \text{ XOR } x_0$

BONUS !!!!!!
HAMMING CODE

```
module hamming_decoder(
    input  wire [6:0] r,        // (4 data + 3 parity)
    output reg  [6:0] fixed,    // Corrected output
    output wire [3:0] data_out,
    output wire [2:0] syndrome
);

    // Get Syndrome Bits
    assign syndrome[0] = r[0] ^ r[2] ^ r[4] ^ r[6];
    assign syndrome[1] = r[1] ^ r[2] ^ r[5] ^ r[6];
    assign syndrome[2] = r[3] ^ r[4] ^ r[5] ^ r[6];

    // Correct the error if syndrome is not zero
    always @(*) begin
        fixed = r; // Deafault = correct

        if (syndrome != 3'b000) begin
            fixed[syndrome - 1] = ~r[syndrome - 1];
        end
    end

    assign data_out = {fixed[6:5], fixed[4], fixed[2]};

endmodule
```

BONUS !!!!!!
HAMMING CODE

```
module hamming_encoder (
    input  wire [3:0] d,     // 4 data bits
    output wire [6:0] c      // 4 data + 3 parity
);

    /*
    Blah blah blah
    */


    // Data bits
    assign c[2] = d[0]; // D1
    /*
    (...)
    */
    assign c[6] = d[3]; // D4

endmodule
```